

Person 5 — Backend / Database / Security / Deployment

Intelligent Land Record Digitization, Validation & Urban Cadastral Mapping Platform |
Step-by-step implementation specification | Unified Master Blueprint v2.0

Assigned role: Person 5 — Backend / Database / Security / Deployment

Document set: Unified Master Blueprint v2.0

Scope: SIH 12 + SIH 18 combined platform

Canonical rule: This document must not introduce a technology, endpoint, role, state, table, or handoff that conflicts with the Master Blueprint.

1. Mission

Own the secure platform foundation: REST APIs, PostgreSQL/PostGIS, object storage, asynchronous jobs, authentication, RBAC, project-scoped authorization, audit, observability, migrations and deployment.

2. Non-negotiable shared rules

- Use the canonical stack, API paths, entities, statuses, application roles and event names from Master Blueprint v2.0.
- Do not change another person's contract by silently renaming fields or endpoints.
- All work is project-scoped through `project_id`.
- All long-running work is asynchronous with job/workflow status.
- Original documents/source imagery remain immutable.
- Every AI output carries confidence, provenance, `model_version` and timestamp.
- Every critical mutation is auditable.
- Before handoff, run tests and provide evidence.

3. Exact handoff principle

A handoff is complete only when the receiving person can consume the documented output without guessing field names, endpoint names, status values, storage locations, or semantics.

4. Database foundation

```
users(id, email, password_hash, full_name, is_active, created_at)
roles(id, name)
permissions(id, code)
user_roles(user_id, role_id)
role_permissions(role_id, permission_id)
projects(id, name, description, state, owner_id, created_at, updated_at)
project_members(project_id, user_id, role, created_at)
files(id, project_id, original_name, mime_type, size_bytes, sha256, storage_key, status)
processing_jobs(id, project_id, job_type, status, idempotency_key, progress, retry_count, error_json)
audit_logs(id, actor_id, project_id, action, target_type, target_id, before_hash, after_hash, metadata_json, created_at)
```

5. Step-by-step backend implementation

- Create Pydantic settings and `.env.example`; never hard-code secrets.
- Create PostgreSQL/PostGIS connection and Alembic migrations.
- Create canonical users, roles, permissions, project and file tables.
- Create document, extraction, validation, review, imagery and GeoAI tables required by the master schema.
- Seed application roles: ADMIN, OFFICER, REVIEWER, SURVEYOR, VIEWER.
- Seed permissions from the canonical permission list; implement `role_permissions`.
- Implement secure authentication and token refresh/logout; support an approved government identity provider adapter where required.
- Implement backend authorization dependencies for role and project scope.
- Implement private object storage with short-lived signed upload/download URLs.
- Validate file MIME/content, size, extension, checksum, safe filenames and project ownership.
- Implement asynchronous jobs with idempotency keys, retries, timeout, cancellation and dead-letter/failure status.
- Implement audit middleware plus explicit mutation audit events.
- Add request correlation IDs, structured logs, rate limits, secure headers, CORS policy and secret handling.
- Implement health/readiness and metrics endpoints.

- Generate OpenAPI from the actual FastAPI schemas and use it as the frontend contract source.
- Implement consistent error responses with `request_id`.
- Write auth/RBAC/project-isolation/security/migration/API contract tests.
- Create Docker Compose for frontend, backend, worker, postgres-postgis, redis, minio, geoserver and optional monitoring.

6. RBAC enforcement

The backend must enforce both **role permission** and **project scope**. A user may possess a permission globally but still be denied if the target project is outside the user's project membership.

```
authorize(user, permission, project_id):
    1. authenticate user
    2. load active roles
    3. check permission through role_permissions
    4. if resource is project-scoped, verify project membership/scope
    5. deny with 403 if any check fails
    6. record sensitive access/mutation as required by audit policy
```

7. Storage security

- Browser requests upload authorization → backend presign → private object storage.
- Browser requests download → backend authorization → short-lived signed URL.
- Never expose bucket credentials to frontend.
- Reject unsafe object keys/path traversal.
- Preserve original object and processed derivatives as separate references.
- Expire signed URLs and test expiration.

8. Security acceptance tests

- Viewer cannot access draft records.
- User cannot access another project's files.
- Expired signed URL fails.
- Invalid MIME/content is rejected.
- Repeated idempotency key returns same job.
- Worker retry does not duplicate records.
- Audit logs exist for upload, correction, approval, rejection, export and permission changes.
- Backup/restore procedure is documented and tested.

9. Deployment checklist

- Fresh machine builds containers.
- Migrations apply cleanly.
- Seed roles/permissions load correctly.
- Health and readiness are green.
- Storage bucket is private.
- Secrets come from environment/secret manager.
- OpenAPI is generated.
- Worker retries work.
- Audit records persist.

- CI passes.

10. Handoff

- P1 receives workflow/job APIs, authorization hooks and integration event persistence.
- P2/P3/P4 receive service invocation contracts, storage access patterns and database schemas.
- P6 receives OpenAPI and role/permission responses; frontend must not guess permissions.

11. Definition of done

- Canonical schema/migrations exist.
- RBAC and project isolation pass tests.
- Private storage and signed URLs pass tests.
- Jobs/retries/idempotency pass.
- Audit and observability pass.
- Docker Compose starts the system.
- OpenAPI is the actual API contract.